

ExitAI — Project Documentation

AI-assisted employee offboarding · LangGraph hub-and-spoke multi-agent system over Supabase

Generated 19 September 2026 from the actual repository at C:\Users\Aswin.v\Desktop\Exit Ai — git branch main, HEAD 1d5067c. Every statement below was checked against source files, SQL migrations, or a live query against the Supabase project. Where something is manual, mocked, or demo-scale, it is labelled as such.

Contents

1. Project overview	7. File / code map
2. Technology stack	8. Database and security model
3. What RAG is, in this project	9. The five roles
4. Architecture — hub and spoke	10. Honest status — what is real, manual, or mocked
5. All 24 agents	11. LLM vs. deterministic agents — who calls the AI
6. Agent workflow, step by step	12. The actual system prompts
	13. Q&A — 27 questions a manager or director might ask

Read this first (the honesty summary). Everything in sections 1–9 is built and runs against a live Supabase project with real data (107 profiles, 27 exit cases, 316 tasks, 323 agent-run records). Four things are *not* fully production-real, and are called out again in place:

- **No scheduler exists anywhere in the repo.** Every "periodic"/"scheduled" agent (#9 SLA escalation, #14 analytics, #17 optimizer, #19 trend analyst, #22 policy auditor, #23 attrition) is run by hand from the command line. There is no cron, no pg_cron, no GitHub Action, no timer.
- **#15 Multi-System Clearance is a demo stand-in.** No external IT-asset-management, HRMS, or finance system exists. Its three adapters query *our own* Supabase tables through a real adapter contract (timeouts, retries, status codes) so the shape is production-ready, but the "external systems" are ours.
- **#18's execution step uses a MockITAdapter.** Plan generation and human approval are real; the "execute" call does not talk to Okta/Jamf/Google Workspace — nothing real is disabled.
- **Phase 9 (full real-browser end-to-end review) is not signed off.** Individual flows have been Playwright-verified; the single, complete, all-five-roles browser pass is still open (item E1 in blueprint1.md).

1. Project overview

What ExitAI is

ExitAI is a web application that runs the whole process of an employee leaving a company — "offboarding" or "exit clearance" — with a team of AI agents doing the coordination work that a person would otherwise chase by email.

An employee submits their resignation in the app. From that moment the system generates their personal exit checklist, tells their manager, tracks knowledge handover, generates the IT deprovisioning plan, verifies compliance (NDA, assets, access), waits for Finance to settle dues, assesses risk and rehire eligibility from the exit interview, and finally issues the relieving letter. Five people see five different dashboards, each showing only what their role is responsible for.

The problem it solves

Problem with manual offboarding	What ExitAI does about it
Exit checklists are copy-pasted and generic, so role-specific handover gets missed.	An LLM generates the checklist from the employee's actual role and department, with due dates derived from their last working day.
Nobody knows who is currently blocking a clearance. Chasing happens over email and gets dropped.	Every task has a stage, an owner and a due date in one database. The SLA agent names the specific blocker, how many days late, and the downstream impact.
An employee "leaves" but their accounts stay live for weeks because IT was told too late.	The IT deprovisioning plan is generated automatically the moment the manager signs off knowledge transfer — not after someone remembers to raise a ticket.
Final clearance gets signed while the NDA is unreturned or a laptop is missing, and no one notices until audit.	A compliance agent blocks final clearance and names the exact missing item. It cannot be bypassed in the UI, because the relieving-letter gate is enforced by a database function.
Exit interviews are filed and never read, so the same attrition cause repeats.	Each interview is summarised with sentiment and themes; a longitudinal agent counts themes across all interviews and raises a rising-theme alert per department.
Exiting employees keep asking HR the same questions about notice period, settlement, and documents.	A RAG chatbot answers from the company's actual exit policy document, with citations, and refuses to invent company-specific facts.
Sensitive assessments (risk score, rehire eligibility, interview sentiment) leak to managers.	Those columns are HR-only and enforced in the database with row-level security plus restricted views — not merely hidden in the UI.

Scale of the running system (live query, 17 Sep 2026)

Table	Rows	Table	Rows
profiles	107 (100 employees, 2 HR, 2 manager, 2 IT, 1 finance)	agent_runs	323
exit_cases	27	compliance_checks	78
exit_tasks	316	analytics_insights	18
exit_interviews	18	kt_reviews	27
exit_docs (RAG chunks)	67	case_documents	13
trend_alerts	4	—	—

2. Technology stack

Every technology listed here is actually used by code in this repository. Versions come from `package.json` and `agents/requirements.txt`.

2.1 Frontend

Technology	What it is	How it is used here / why chosen
React 18.3	UI library — builds the page out of components that re-render when data changes.	All five dashboards. Chosen because the dashboards are data-driven tables and cards that must refresh the instant a button writes to Supabase.
Vite 6	Dev server and bundler.	<code>npm run dev</code> / <code>npm run build</code> . Chosen for instant hot reload and because its <code>import.meta.env.VITE_*</code> convention makes the "only the anon key ever reaches the browser" rule mechanical — a variable without the <code>VITE_</code> prefix physically cannot enter the bundle.
React Router 7.18	Client-side routing.	<code>src/routes/AppRoutes.jsx</code> — one route tree per role, each guarded by <code>role === '...'</code> so an IT user typing <code>/hr</code> is redirected, not served.
@supabase/supabase-js 2.116	Official Supabase client.	Auth, table reads/writes, RPC calls, Storage uploads, and Edge Function invocation — one library for all of it. Created once in <code>src/lib/supabase.js</code> with the anon key only.
Tabler Icons (webfont) 3.31	Icon set delivered as a font.	Dashboard iconography. A webfont avoids shipping an icon-component library.
CSS design tokens	Plain CSS custom properties.	<code>src/styles/tokens.css</code> + <code>dashboards.css</code> . No Tailwind, no CSS-in-JS — the approved design is a fixed token set, so a token file is the smallest thing that keeps five dashboards visually identical.
Playwright 1.63 dev only	Headless real-browser automation.	Verification scripts in <code>scripts/*.cjs</code> log in as each role and assert against a real Chromium, because "a script passed once" is not accepted as proof for UI work in this project.

2.2 Backend — Supabase

Technology	What it is	How it is used here / why chosen
PostgreSQL	The relational database behind Supabase.	10 tables, 5 views, 6 functions, in 28 numbered migrations under <code>supabase/migrations/</code> . Chosen because the data is inherently relational (cases → tasks → documents) and because Postgres can enforce the access rules itself.
Supabase Auth	Email/password authentication issuing JWTs.	<code>profiles.id</code> is a foreign key to <code>auth.users.id</code> , so <code>auth.uid()</code> joins straight into every security policy. Login is a single <code>signInWithPassword</code> call.
Row Level Security (RLS)	Per-row access rules enforced by Postgres on every query.	The core of the security model — see §8. Chosen over checking permissions in the frontend because the frontend holds a key any user can read out of the bundle; the database cannot be argued with.
pgvector	Postgres extension adding a <code>vector</code> column type and similarity operators.	The RAG store — <code>exit_docs.embedding</code> <code>vector(1536)</code> with an HNSW cosine index. See §3.

Supabase Storage	S3-style object storage with the same RLS engine.	Private exit-documents bucket. Employees upload to <code><case_id>/<doc_type>-<timestamp>.<ext></code> ; a storage policy checks the first path segment is a case they own.
Edge Functions (Deno/TypeScript)	Serverless functions running in Supabase's cloud.	Three of them — <code>ask</code> , <code>submit-resignation</code> , <code>forward-to-hr</code> . Used wherever a secret must never reach the browser but the operation must be callable from it.
Python agent service	A local HTTP bridge, <code>agents/service.py</code> , on <code>localhost:8787</code> .	Edge Functions run in Supabase's cloud and have no network path to the machine where the agents run. The browser (which <i>is</i> on that machine in dev) calls this service directly. Built on stdlib <code>http.server</code> — not Flask or FastAPI — because neither is a project dependency and this is nine routes.

2.3 Agent framework and LLM layer

Technology	What it is	How it is used here / why chosen
LangGraph ≥ 0.2	Framework for building agents as a <i>graph</i> of nodes with typed shared state and conditional edges.	Every agent is a compiled <code>StateGraph</code> , and the Supervisor is a graph whose nodes are those agents. Chosen over a plain function chain because the manager gate needs a real conditional branch (<code>approve</code> → IT, <code>reject</code> → escalate) and because a graph gives one uniform "invoke" shape for every agent. <i>Not CrewAI</i> .
Portkey AI gateway	An LLM proxy: one endpoint and one key in front of many model providers, with virtual keys, routing, and observability.	Self-hosted at <code>https://portkeygateway.perficient.com</code> . Both the Python agents and the Edge Functions call it. Chosen so the model can be swapped by changing one environment variable, and so no provider key is ever held by application code.
Azure OpenAI gpt-5.6-sol	The generation model, reached as <code>@azure-openai-eus2/gpt-5.6-sol</code> .	Writes checklists, reviews KT documents, summarises interviews, and drafts narratives. Migrated from Claude: the code still speaks the <i>Anthropic Messages</i> wire protocol (<code>POST /v1/messages</code> , <code>anthropic-version: 2023-06-01</code>) and the env vars are still named <code>ANTHROPIC_*</code> , but Portkey translates that request to Azure OpenAI and translates the reply back. So the names describe the <i>protocol</i> , not the vendor — worth knowing, because a reviewer reading <code>ask_claude()</code> will otherwise assume Claude is being called.
OpenAI text-embedding-3-small	The embedding model, 1536 dimensions, reached through Portkey's virtual key.	Turns policy chunks and user questions into vectors. 1536 is hard-coded into the <code>exit_docs.embedding</code> column and re-checked by the ingest script, so a model swap fails loudly instead of silently corrupting the index.

Two consequences of the Claude → Azure migration that show up in the code.

- (1) The gateway model has extended thinking enabled, so `content[0]` is sometimes a *thinking* block with no text. `agents/core/llm.py` searches for the first block of `type: "text"` instead of assuming index 0, and raises a clear error naming `stop_reason` when a reasoning model spends its whole budget before emitting text.
- (2) The new model emits typographic apostrophes (don't) where the old one emitted ASCII, which broke the /ask refusal detector. It now normalises the apostrophe first.

2.4 Everything else

Technology	What it is	How it is used here / why chosen
Tesseract OCR (via pytesseract + Pillow)	Open-source optical character recognition — reads text out of an image.	Real OCR, not simulated. <code>agents/spokes/doc_collection.py</code> downloads an uploaded document from Storage, runs <code>pytesseract.image_to_string</code> against the actual Tesseract binary, then keyword-checks the extracted text (an NDA must contain both "non-disclosure/confidential" <i>and</i> "signature/signed", plus the employee's own name and a date). Chosen over a vision LLM because the checks are deterministic substring rules — an LLM would add cost, latency and a hallucination risk for a task that is exact matching. Binary path falls back to the <code>TESSERACT_PATH</code> env var when not on <code>PATH</code> .
Google Calendar API (OAuth 2.0, <code>google-api-python-client</code>)	Calendar event creation.	Real events are created. When the HR agent writes KT tasks, <code>calendar_booking.py</code> books an event per task and stores the Google event id back on <code>exit_tasks.kt_event_id</code> . demo-scale One shared demo Google account authorised once via <code>agents/oauth_setup.py</code> holds every booking — there is no per-user Google login. Scope is <code>calendar.events</code> only (least privilege). If the OAuth vars are unset, bookings are logged and never raise.
Gmail SMTP (<code>smtplib</code> / <code>denomailer</code>)	Outbound email over <code>smtp.gmail.com:465</code> with an app password.	Real mail is delivered. Python side uses <code>stdlib smtplib</code> ; the forward-to-hr Edge Function uses <code>denomailer</code> against the same account. Sends always come from <code>aswindevv2005@gmail.com</code> and go to the recipient's own address from <code>profiles.email</code> — never redirected to one test inbox. dev gate If <code>EMAIL_TEST_RECIPIENT</code> is unset, every send degrades to a printed dev-log line instead of a network call. It is currently set, so real sends are on.

python-dotenv, requests, typing_extensions	Env loading, HTTP, typed dicts.	Supporting libraries for the agent service. <code>load_dotenv(..., override=True)</code> is deliberate — a stale Windows user-level <code>ANTHROPIC_BASE_URL</code> was shadowing the correct value.
---	---------------------------------	---

Small drift worth owning if asked. `agents/requirements.txt` lists seven packages but omits `pytesseract` and `Pillow`, which `doc_collection.py` imports. They are installed in the venv, so it runs — but a fresh `pip install -r requirements.txt` would not reproduce the OCR agent.

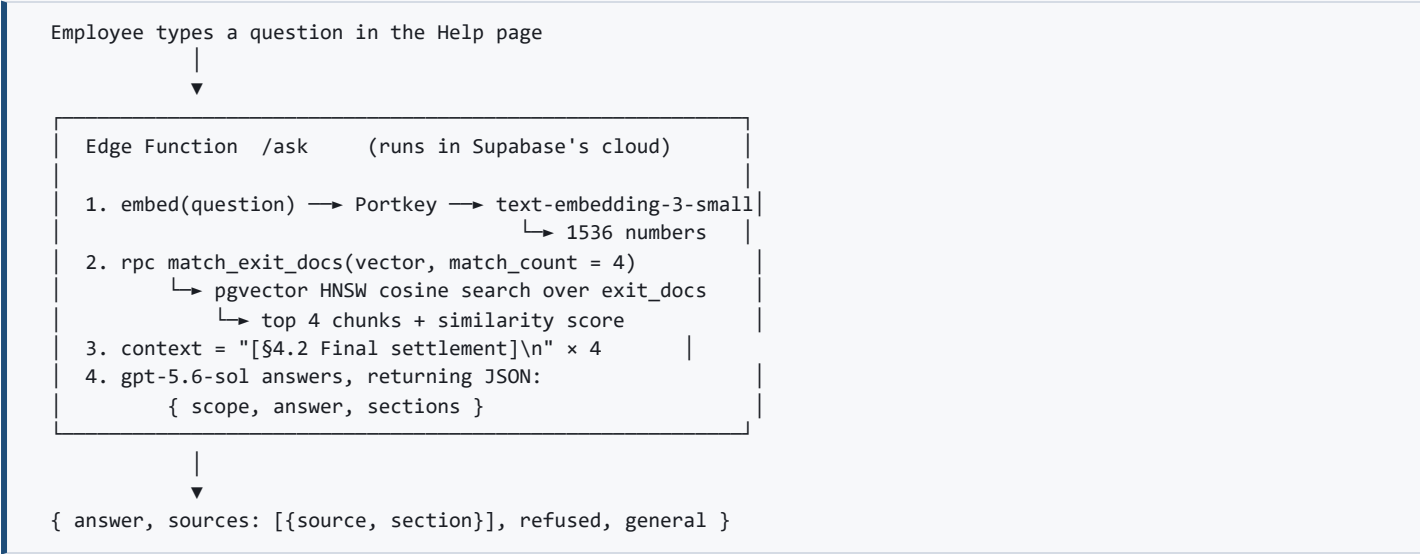
3. What RAG is, in this project

The plain-language version

A language model does not know your company's exit policy. You could paste the whole policy into every question, but that is expensive and the model loses the thread in long text. **RAG — Retrieval-Augmented Generation** — is the cheap fix: keep the policy in a database, fetch only the two or three passages that actually relate to the question, and hand just those to the model along with the question. The model answers from what it was handed, and you can show the employee which sections the answer came from.

The five steps, and exactly what each one is here

Step	What happens	The concrete implementation in this repo
1. Chunk	Split the document into passages small enough to be precise.	<code>scripts/ingest_docs.js</code> reads <code>scripts/exit_policy.md</code> and splits on section headings — a new chunk starts at every <code>§n.n Title</code> line, and following paragraphs attach to it. One chunk = one policy section , deliberately, so a citation can never point at section 4.1 for text that actually came from 4.2. Result: 67 chunks currently in the table.
2. Embed	Turn each chunk into a list of numbers (a vector) that encodes its meaning.	<code>POST {PORTKEY_BASE_URL}/v1/embeddings</code> with OpenAI text-embedding-3-small (1536 dimensions) via the Portkey virtual key. The script asserts <code>embedding.length === 1536</code> before inserting, so a model change fails on ingest rather than at query time.
3. Store	Save chunk text + vector so you can search by meaning.	Table exit_docs — <code>id</code> , <code>source</code> , <code>section</code> , <code>content</code> , <code>embedding vector(1536)</code> — with an HNSW index using <code>vector_cosine_ops</code> . HNSW was chosen over ivfflat because ivfflat needs training data and returns poor results until the table is large; HNSW works correctly from the very first row. Ingest is idempotent: it deletes prior rows for that source before re-inserting.
4. Retrieve	Embed the question the same way, find the nearest chunks.	SQL function <code>match_exit_docs(query_embedding vector(1536), match_count int default 4)</code> (migration <code>0003_rag.sql</code>). It orders by pgvector's cosine-distance operator <code><=></code> and returns <code>1 - distance</code> as a similarity score. The app requests top-k = 4 .
5. Generate	Give the model the retrieved text plus the question; it answers from that.	Edge Function supabase/functions/ask . Builds the prompt as <code>[Section title]\ncontent blocks</code> , calls <code>gpt-5.6-sol</code> through Portkey, and returns <code>{ answer, sources, refused, general }</code> to the browser.



What makes this implementation careful rather than naive

- **The model reports its own citations as data, not prose.** It must return a JSON object with a `sections` array listing the section titles it used. The UI renders the citation line from that array. A naive version lets the model write "Sources: ..." into its answer, which then disagrees with whatever the UI lists separately.

- **Three-way scope, not two.** `scope` is policy (the context answers it), general (not in the policy, but it is a reasonable work question — give general guidance and *never* state a company-specific number as if it were this company's rule), or refused (nothing to do with work — one sentence, contact HR). This distinction matters because an employee will act on a notice period or a settlement figure; an invented one is the failure that actually hurts.
- **A similarity floor.** Below cosine similarity **0.35** the top chunk is not genuinely relevant (greetings, chit-chat), so the answer is still returned but *no citations are attached* — unrelated sections are never shown as if they grounded it.
- **A refusal fallback path.** If the model ignores the JSON contract, a wording heuristic classifies "I don't have..." as a refusal, so the employee still gets the *Forward to HR* button rather than a dead end.
- **Forward-to-HR never silently fails.** If Gmail SMTP times out (8 s cap), the `forward-to-hr` function writes the question to `agent_runs` with `status='delivery_failed'` and tells the employee the truth — that it is logged and may be delayed — instead of showing a generic error.

Why pgvector instead of a dedicated vector database

Reason	Detail
One database, one backup, one set of credentials	Pinecone/Weaviate/Qdrant would be a second managed service with its own key, SDK, billing, uptime and backup story. The corpus here is 67 chunks of one policy document — that does not justify a second system.
Security actually composes	This is the decisive reason. <code>exit_docs</code> has RLS enabled with <i>zero</i> policies, so the browser's anon key reads nothing from it. And <code>match_exit_docs</code> is deliberately not <code>SECURITY DEFINER</code> , so it runs with the caller's rights and inherits that same denial. The only path to the corpus is the service-key <code>/ask</code> function — which makes <code>/ask</code> a genuine security boundary where questions can be logged, rate-limited and redacted, not a convenience wrapper around a table the browser could already read. An external vector DB would sit outside Postgres' permission model entirely, and you would have to rebuild that boundary by hand.
No sync problem	A separate vector store must be kept consistent with the relational data. Here a chunk is a row in the same database as everything else; there is nothing to drift.
Joins are free	A retrieved chunk can be joined to any other table in the same query. Across a network boundary that becomes application-side stitching.
HNSW is good enough at this scale	Dedicated vector DBs earn their keep at millions of vectors with sharding and filtered search. At 67 — or 67,000 — a Postgres HNSW index answers in single-digit milliseconds.

4. Architecture — hub and spoke

The pattern

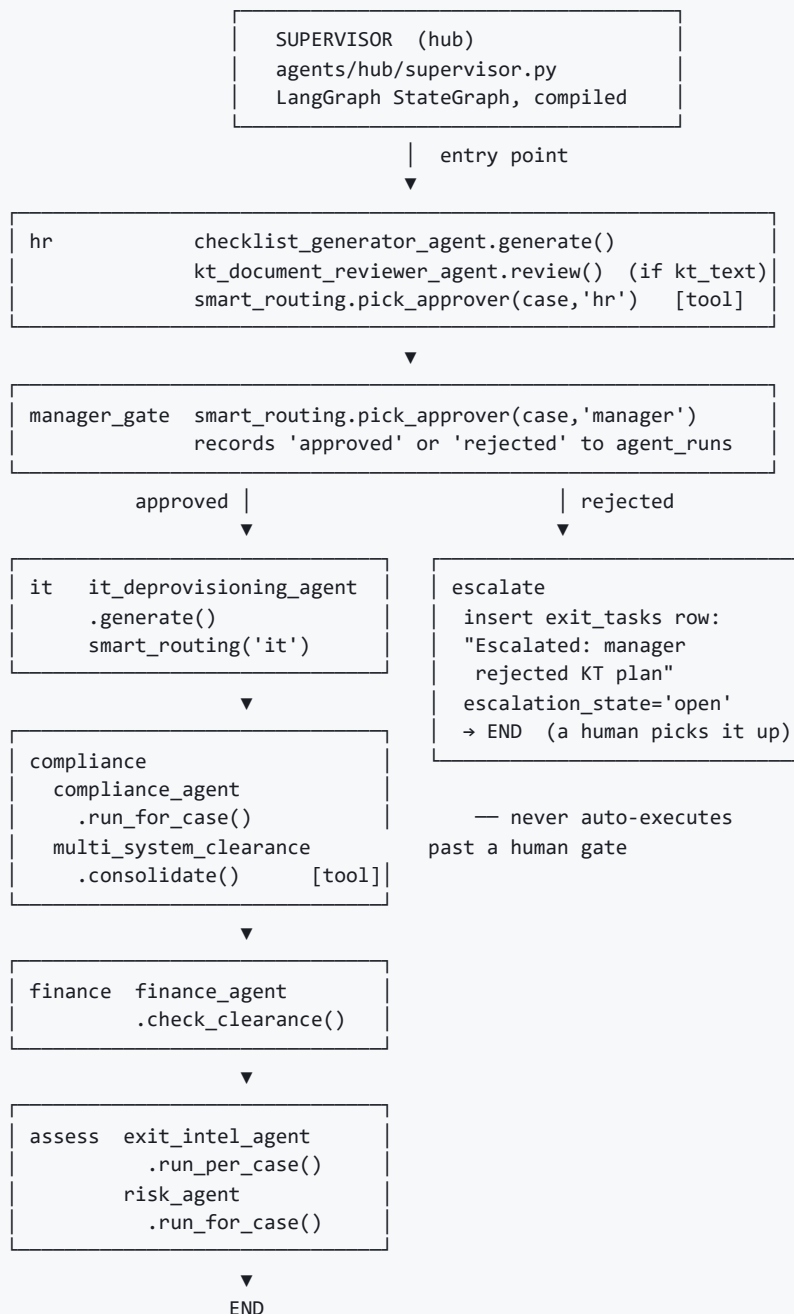
Hub-and-spoke means one *Supervisor* agent (the hub) owns the case and decides which specialist agent (a spoke) runs next. **Spokes never talk to each other.** The IT agent does not call the Finance agent; it returns to the hub, and the hub decides.

This is a deliberate choice over letting agents call each other freely. With one hub there is exactly one place that knows the order of stages, one place that handles a rejection, and one ordered trace to read when something goes wrong. Free-form agent-to-agent messaging gives you emergent loops and no single point to reason about.

Naming note, in case it comes up. "Hub-and-spoke" here is an *architecture pattern*. It has nothing to do with HubSpot — there is no CRM anywhere in this project. Likewise the agent framework is **LangGraph**, not CrewAI.

The Supervisor graph, as actually coded

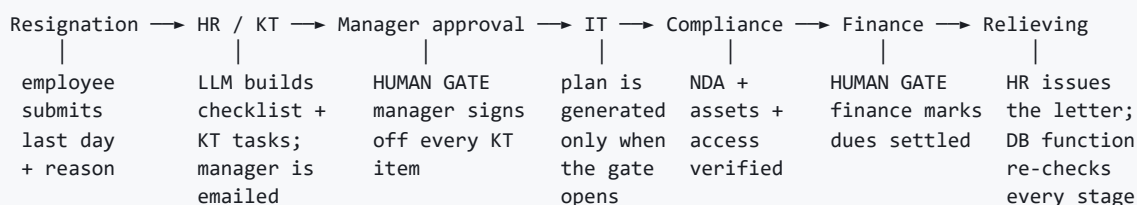
`agents/hub/supervisor.py` builds a `LangGraph StateGraph` over a typed state `{ case_id, kt_text, interview_text, simulate_rejection, log }`:



Two things in that diagram are worth naming explicitly:

- **Tool calls vs. graph nodes.** `smart_routing` (#11) and `multi_system_clearance` (#15) are *not* top-level nodes. They are invoked from inside the `hr/manager/it` and `compliance` nodes respectively — the same way a tool is called from within a step, rather than being a step.
- **Escalation does not loop back.** A rejected gate writes a visible task row and ends the graph. It does not retry or re-route itself. That is the same posture as the IT agent never writing `status='done'`: *an agent proposes, a human disposes*.

The business flow the graph implements



Two ways the pipeline is driven

Path	Entry point	Behaviour
CLI / demo	<code>python -m agents.run_case <case_id></code>	Runs the <i>whole</i> supervisor graph in one go, streaming the live trace. The manager gate is auto-approved (or auto-rejected with <code>--reject</code>). This is the path you run in a terminal to show the hub→spoke→hub handoffs.
Browser / real	Nine routes on <code>agents/service.py</code> (localhost:8787)	Deliberately not the full graph. Running the whole graph on resignation would auto-approve the manager gate and complete IT, finance and risk the instant a case is opened — finishing an exit nobody has reviewed. Instead each real human action re-enters the pipeline at its own stage.

Why the browser path needed a /manager-approve route. Stage-gating only works if every gate can *open* later. The rejection branch had a trigger (`/reject-manager-task`); the approval branch had none. A browser-created case therefore got its HR checklist and stopped forever: nothing outside the graph ever called `it_agent.generate_plan`, so no `stage='it'` rows were created, compliance sat on "no IT task found", and the relieving gate could never open. That was a real bug, fixed by adding the mirror route. It never performs the approval itself — it re-reads the case with the service key and advances only when every non-escalation manager task is done and no escalation is open.

Observability — agents are visible, by design

Every agent node is wrapped in `@traced_node` from `agents/core/trace.py`, which prints live to the terminal as it runs: entry with inputs, every LLM call with model, latency and token counts, every DB write, then output and elapsed time — indented by call depth so hub→spoke→hub reads in order. Output is flushed per line so it appears as it happens, colours auto-disable when piped to a file, and field names in a redact list (`password`, `ssn`, `email`, `token`, `api_key`) are masked. `TRACE_LEVEL` switches between `full` | `summary` | `off`.

```
└ Supervisor -- HR stage ▶ start
└ inputs: {"case_id": "13eaa5fe-...", "simulate_rejection": false, "log": []}
└ Checklist Generator agent (#5) ▶ start
└   ↳ llm @azure-openai-eus2/gpt-5.6-sol 2.41s in=412 out=98 tok
└   ↳ db insert exit_tasks (7 rows)
└ Checklist Generator agent (#5) ✓ done 2.63s
└ ↳ db insert agent_runs (1 rows) hr: checklist + kt-review done
└ Supervisor -- HR stage ✓ done 3.02s
```

Because the terminal trace is stdout only, the same stage transitions are also persisted to the `agent_runs` table, which is what the HR dashboard's *Agent activity* page reads — so a non-terminal user can still see that the agents ran.

5. All 24 agents

The company brief (docs/agent_requirements.md) specifies 24 capabilities. Those are delivered by **~14 Python modules plus one Edge Function** — several capabilities are different *modes or triggers* of one underlying agent rather than duplicated code. Where a capability is a thin named wrapper over another module, the table says so; that is the project's stated build rule ("reuse existing verified logic — wrap, don't duplicate").

Status legend: WORKING real logic, real data, verified · MANUAL works, but only when a human runs it — no scheduler · MOCK / DEMO-SCALE the external dependency is a stand-in · HUMAN a person, not an LLM agent.

#	Agent	File	What it does	Reads / writes	Trigger	Real status
1	Manager	service.py routes + ManagerPages.jsx	The manager approval gate. Reviews each KT task, then signs one clearance per employee. Approve advances the case to IT; Reject opens an escalation for HR.	Reads manager_case_view, exit_tasks. Writes exit_tasks.status, agent_runs.	Human click → /manager-approve or /reject-manager-task	HUMAN GATE — deliberately not an LLM
2	HR Agent	hr_agent.py	Two LangGraph subgraphs. <i>checklist</i> : role + department → 2–4 HR tasks and 2–4 manager KT tasks, with due dates at last-working-day –3 and –1 days. <i>kt-review</i> : reads a handover document and flags gaps. Fires the KT reminder email and books calendar events.	Reads exit_cases, exit_tasks. Writes exit_tasks, kt_reviews.	Per case — via supervisor, /activate-exit, or CLI	WORKING
3	IT Agent	it_agent.py	LLM generates the deprovisioning plan as stage='it' tasks. Prompt is tuned so titles hit the IT dashboard's own keyword buckets (identity/SSO, source control, assets, SaaS). Never writes status='done' — it only proposes.	Reads exit_cases, exit_tasks. Writes exit_tasks.	Supervisor it node, or /manager-approve once every KT task is signed. Idempotent.	WORKING
4	Finance Agent	finance_agent.py	Deterministic clearance gate — no LLM . Clears only when hr + manager + it tasks are all done <i>and</i> exit_cases.finance_cleared is true. Otherwise blocks with the reason "dues/settlement not confirmed". Fires the completion email on the pending→done transition.	Reads exit_cases, exit_tasks. Writes exit_tasks.	Supervisor finance node, or /finance-settle-check	WORKING

5	Exit Checklist Generator	checklist_generator_agent.py	Named, individually traced wrapper over <code>hr_agent.generate_checklist</code> . Exists so the capability appears as its own line in the trace. Does not reimplement anything.	(delegates to #2)	Supervisor hr node	WORKING (wrapper)
6	Email Drafting	email_drafting_agent.py over notifications.py	The single routing point for all seven email types: KT reminder, overdue warning, resignation notice, completion notice, relieving-letter notice, SLA escalation, document reminder. Records every send to <code>agent_runs</code> with a real outcome — <code>smtp_accepted</code> , <code>dev_logged</code> , or <code>failed</code> , never a fabricated delivery claim.	Reads <code>profiles</code> , <code>exit_cases</code> . Writes <code>agent_runs</code> .	Event-driven — called by whichever agent reaches that stage	WORKING — real Gmail SMTP. Templated, not LLM-written
7	FAQ Chatbot (RAG)	supabase/functions/ask	Employee Q&A over the exit policy. Embeds the question, retrieves top-4 chunks from pgvector, answers with citations, and classifies each answer as <code>policy</code> / <code>general</code> / <code>refused</code> . See §3.	Reads <code>exit_docs</code> via <code>match_exit_docs</code> . Writes nothing.	Employee types a question (Help page)	WORKING — deployed Edge Function
8	Exit Interview Summarizer	exit_intel_agent.py (per-case mode)	LLM turns interview text into summary, sentiment, themes, rehire eligibility and a rationale. Two-node subgraph: <code>analyze</code> → <code>persist</code> .	Reads <code>exit_cases</code> . Writes <code>exit_interviews</code> .	Per case — employee submits the interview form → <code>/submit-exit-interview</code> ; also the supervisor assess node	WORKING
9	SLA Escalation	sla_escalation.py	Scans every pending task past its due date by ≥ 5 days , and composes an escalation naming the specific blocker, the days overdue, and the downstream impact. Sends to the <i>blocker</i> (HR or the manager), not the employee. Pure arithmetic — no LLM.	Reads <code>exit_tasks</code> , <code>exit_cases</code> , <code>profiles</code> . Writes <code>agent_runs</code> ; sends email.	<code>python -m agents.analytics.sla_escalation</code>	MANUAL — the brief says "monitors"; nothing schedules it
10	KT Document Reviewer	kt_document_reviewer_agent.py	Named wrapper over <code>hr_agent.review_kt_document</code> . LLM checks a handover document for completeness by role, and splits the output: neutral actionable gap tasks the employee can see, evaluative summary into HR/manager-only <code>kt_reviews</code> .	(delegates to #2) Writes <code>exit_tasks</code> , <code>kt_reviews</code> .	Supervisor hr node, when KT text is supplied	MANUAL INPUT — there is no KT upload in the UI . The Knowledge-transfer page is read-only; document text is passed as a file path from the CLI

11	Smart Routing	smart_routing.py	Picks the real approver profile for a stage, preferring a department match and skipping anyone with <code>out_of_office = true</code> in favour of a real delegate account. Verified live: setting Siva out-of-office routed to "Divya (HR Delegate)"; reverting routed back.	Reads profiles, exit_cases. Writes agent_runs.	Tool call from the hr / manager / it supervisor nodes	WORKING — but see note ► the department-match branch never fires today
12	Exit Risk Assessment	risk_agent.py	Deterministic weighted score, no LLM : tenure 0.2, department criticality 0.3, interview sentiment 0.3, outstanding tasks 0.2. ≥ 0.66 high, ≥ 0.4 medium, else low. Also derives <code>rehire_eligible</code> .	Reads exit_cases, profiles, exit_interviews, exit_tasks. Writes exit_cases (HR-only columns).	Supervisor assess node; or CLI per case / <code>--all</code>	WORKING — but tenure is a proxy ► see note
13	Compliance Verification	compliance_agent.py	Verifies NDA, asset return and access revocation before final clearance — by deterministic keyword match over tasks or an OCR-validated case_documents row. Blocks by leaving a <code>stage='compliance'</code> task pending with the exact missing items named. Idempotent (updates the same row). Excludes its own summary task from the match so it cannot block on itself.	Reads exit_tasks, case_documents, exit_cases, agent_runs. Writes exit_tasks, compliance_checks.	Supervisor compliance node; also re-run by four service routes after each human action	WORKING — 78 real check rows
14	Dashboard Insights	analytics_agent.py	Three-node subgraph: aggregate → narrate → persist. All arithmetic is Python Counter work over real rows; the LLM only writes the prose. This is a hard project rule — counts in a narrative must match a manual SQL count.	Reads exit_cases, exit_tasks. Writes analytics_insights (agent_type='dashboard_insights', 5 rows live).	<code>python -m agents.analytics.analytics_agent</code>	MANUAL — the brief says "weekly"; nothing schedules it
15	Multi-System Clearance	multi_system_clearance.py	Consolidates clearance across "IT asset management", "HRMS" and "finance" into one status object via three adapters with a real contract: enforced timeout (<code>ThreadPoolExecutor.result(timeout=...)</code> , not cosmetic), retry, error capture, and CLEARED / PENDING / FAILED / TIMEOUT status.	Reads exit_tasks, case_documents. Writes agent_runs only.	Tool call from the compliance supervisor node	DEMO STAND-IN — the three "external systems" are our own Supabase tables. ► see note

16	Document Collection	doc_collection.py	Three-node subgraph: gather → remind → validate. Required docs are a deterministic Python list (NDA + Asset Return Form, plus Company Asset Declaration for IT/Engineering); "missing" is computed as a set difference at read time. Validation runs real Tesseract OCR on the uploaded bytes and keyword-checks the text, plus the employee's own name and a date.	Reads case_documents, exit_cases, Storage. Writes case_documents, agent_runs; sends reminder email.	Employee upload → /validate-document; full run via CLI	WORKING — OCR is real. Reminder sweep is MANUAL
17	Exit Workflow Optimizer	workflow_optimizer.py	Identifies bottlenecks — which stages and which departments accumulate the most overdue days — and proposes reconfigurations. Reuses <code>sla_escalation.find_breaches</code> for the arithmetic rather than recomputing it.	Reads via #9's breach scan. Writes <code>analytics_insights</code> (<code>agent_type='workflow_optimizer', 4 rows live</code>).	<code>python -m agents.analytics.workflow_optimizer</code>	MANUAL
18	Automated IT Deprovisioning	it_deprovisioning_agent.py	Full plan → approve → execute → verify → audit loop. <i>Generate</i> wraps #3. <i>Approval</i> is the real human click in the IT dashboard. <i>Execute</i> runs each approved task through an adapter, then independently re-derives the expected action so verification can genuinely catch a mismatch rather than re-reading its own write. One <code>agent_runs</code> row per task is both the audit trail and the idempotency guard.	Reads <code>exit_tasks</code> , <code>agent_runs</code> . Writes <code>agent_runs</code> (14 rows live).	IT clicks Approve → /execute-deprovisioning	MOCK EXECUTION — <code>MockITAdapter</code> ; no real Okta/Jamf call, nothing is actually disabled
19	Exit Interview Trend Analyst	exit_intel_agent.py (longitudinal mode)	Counts themes across every interview on file, assigns a severity, and writes rising-theme alerts deduped by (theme, department). Verified live: a re-run inserted "management" and "management support".	Reads <code>exit_interviews</code> , <code>exit_cases</code> . Writes <code>trend_alerts</code> (4 rows live).	<code>python -m agents.spokes.exit_intel_agent longitudinal</code>	MANUAL
20	Exit Process Orchestrator (Supervisor)	supervisor.py	The hub. Compiled LangGraph over the stage nodes, with a conditional edge at the manager gate so a rejection routes to escalation instead of crashing the run. Persists every stage transition to <code>agent_runs</code> so the HR dashboard can show it.	Writes <code>agent_runs</code> , <code>exit_tasks</code> (escalation row). Everything else via the spokes.	<code>python -m agents.run_case <case_id></code>	WORKING

21	Intelligent Rehire Assessment	rehire_agent.py	Wraps the rehire_eligible flag #12 computes, and adds the confidence and evidence the brief asks for, plus its own independent execution record so the assessment is not merely a side effect of the risk write. Rationale comes from exit_interviews.rehire_reason.	Reads exit_interviews. Writes agent_runs.	python -m agents.spokes.rehire_agent <case_id>	MANUAL — 1 run recorded live
22	Policy Compliance Auditor	policy_auditor.py	Three checks per active case, all pure Python against real rows: <i>sla_breach</i> (task ≥ 5 days overdue), <i>missing_approval</i> (case has IT/finance tasks but no agent_runs row recording the manager gate), <i>skipped_step</i> (a finance task exists but compliance never ran). Prints a formatted report and persists it.	Reads exit_cases, exit_tasks, agent_runs. Writes analytics_insights (agent_type='policy_compliance_auditor', 6 rows live), surfaced on HR → Policy audit.	python -m agents.analytics.policy_auditor	MANUAL — the brief says "periodically"; it is on-demand only, nothing imports or schedules it
23	Predictive Attrition	attrition_agent.py	Three-node pipeline matching the brief's Data Collector → Analyzer → Recommender: gathers signals from cases, trend alerts and profiles; identifies at-risk groups; narrates retention suggestions.	Reads exit_cases, trend_alerts, profiles. Writes analytics_insights (agent_type='predictive_attrition', 3 rows live).	python -m agents.analytics.attrition_agent	MANUAL — the brief says "on scheduled triggers"; no scheduler . Risk is identified at <i>department</i> level, not per named individual
24	End-to-End Exit Automation (Capstone)	e2e_automation.py	Composition only — no new agent logic, no new LLM calls. Four nodes: <i>initiate</i> (real service.activate_case), <i>coordinate</i> (the full supervisor graph), <i>SLA check</i> (reuses #9's breach finder, scoped to this one case so it cannot escalate unrelated cases as a side effect), <i>finalize</i> (reads back the persisted compliance/finance rows to decide exit_cases.status).	Reads exit_cases, exit_tasks, profiles. Writes exit_cases.status, agent_runs.	python -m agents.hub.e2e_automation <case_id> (--reject exercises the escalation branch)	WORKING — 3 runs recorded live

Notes the table could not hold — read these before the Q&A

#15 Multi-System Clearance — what "demo stand-in" precisely means. There is one Supabase project here, not three integrated enterprise platforms. So `HRMSAdapter` queries our `exit_tasks` where `stage = 'hr'` (plus the NDA document), `ITAMAdapter` queries `stage = 'it'` (plus asset documents), and `FinanceAdapter` queries `stage = 'finance'`. **Every query is a real live query — no hardcoded status values.** What is stand-in is *which system* is being asked. The adapter contract above them is production-shaped, so swapping in a real HRMS means overriding one `_fetch()` method per adapter and changing nothing else in the file. A live run produced: `it=cleared`, `hrms=pending ["Complete exit interview"]`, `finance=pending ["Clear final settlement dues"]`, `overall=pending`.

#18 IT Deprovisioning — what is mock and what is not. Real: the LLM-generated plan, the human approval click, the RLS-scoped status write, the independent verification logic, and the per-task audit row. Mock: the adapter that would call Okta / Jamf / Google Workspace. The module's own docstring states the reason plainly — *"a task marked done is not proof deprovisioning actually executed"* — so the `execute/verify/audit` layer was built even though the provider behind it is a stand-in. Two fault-injection adapters (`_AlwaysFailAdapter`, `_AlwaysTimeoutAdapter`) exist specifically to prove the verification step can fail.

#11 Smart Routing — the branch that never fires. `Department-match` is real code over a real column, but in the current dataset no HR / manager / IT profile has a non-null department (only employees do), so that branch always falls through to the full candidate pool. The out-of-office branch is exercised and verified. Ordering of "primary" vs "delegate" comes from `profiles.created_at`, i.e. real insert order, not a hardcoded name.

#12 Risk — tenure is a proxy, and it matters. There is no `hire_date` column anywhere in the schema. Tenure risk is derived from `profiles.created_at` instead. Because all seed profiles were created recently, every case currently reads as short tenure (0.9). The weighting and thresholds are real; that one input is not yet. If asked "is the risk score real?" — the arithmetic is, one of its four inputs is a placeholder until onboarding data exists.

#6 Email — "drafting" is templating, not generation. The brief describes LLM-drafted emails. In practice `notifications.py` composes from structured templates with no LLM call, for the same reason the finance agent has none: these are fixed transactional notices where a model adds cost and variance and no value. The `email_drafting_agent` layer adds routing, tracing, and honest outcome recording on top.

6. Agent workflow, step by step

One complete exit case, from the employee clicking Submit to HR issuing the relieving letter. **A = automatic (an agent acts unprompted)**. **H = human gate (nothing proceeds until a person clicks)**.

Step	A/H	Who acts	What happens	What it produces
1	H	Employee	Logs in. <code>EmployeeLayout.jsx</code> checks for an <code>exit_cases</code> row; if there is none the employee is <i>redirected</i> to <code>/employee/resignation</code> and the dashboard is unreachable. They pick a last working day and an optional reason.	A form submission.
2	A	Edge Function <code>submit-resignation</code>	Creates the case. Identity — employee id, name, email, department — is derived server-side from the caller's JWT , never trusted from the request body, so it cannot be spoofed. Manager and HR are looked up. This function deliberately triggers <i>no</i> agent.	One <code>exit_cases</code> row, status open.
3	A	Frontend → <code>/activate-exit</code> → HR Agent (#2, #5)	The browser calls the local agent service. The LLM generates the checklist from role and department; HR tasks get due dates at last-working-day -3, KT tasks at -1. IT-owned titles the model mis-files into <code>manager_tasks</code> are dropped by <code>IT_OWNED_TITLE_RE</code> . Status → <code>in_progress</code> .	<code>exit_tasks</code> rows (hr + manager stages); resignation-notice email to the manager; a Google Calendar event per KT task.
4	H	Employee	Works their checklist: marks HR tasks done, uploads documents (NDA, Asset Return Form, and for IT/Engineering a Company Asset Declaration), completes the exit interview form.	Task status updates; <code>case_documents</code> rows; an <code>exit_interviews</code> row.
5	A	Document Collection (#16)	On each upload the browser calls <code>/validate-document</code> . The agent downloads the object with the service key, runs real Tesseract OCR, and checks the text contains the required keyword groups plus the employee's own name and a date.	<code>case_documents.status</code> → <code>validated</code> or <code>rejected</code> , with the reason.
6	A	Exit Interview Summarizer (#8)	On submit, <code>/submit-exit-interview</code> runs the LLM over the response: summary, sentiment, themes, rehire eligibility and rationale.	<code>exit_interviews</code> enriched. HR-only — the employee's own page shows only "submitted", via a view carrying just case id and timestamp.
7	H	Manager — GATE 1 (#1)	Reviews each KT task (Review / Reject), then signs <i>one</i> clearance per employee. The Sign button only activates once every KT task for that employee is approved. Reject instead writes an escalation row with <code>escalation_state='open'</code> and the case stops.	KT tasks done, or an escalation for HR.
8	A	<code>/manager-approve</code> → IT Agent (#3, #18)	The service <i>re-checks the gate itself</i> with the service key — it advances only when every non-escalation manager task is done and no escalation is open. Then it records the gate to <code>agent_runs</code> (the only source compliance reads for manager approval) and generates the deprovisioning plan.	<code>stage='it'</code> tasks appear in the IT queue, all pending.
9	H	IT	Reviews the plan and approves each item. The agent never marks its own tasks done.	IT tasks → done.

10	A	IT Deprovisioning execute (#18)	After each approval the browser calls <code>/execute-deprovisioning</code> . Each approved task runs through the adapter; the expected action is re-derived independently and compared. The adapter is a mock — nothing real is disabled.	One audit row per task in <code>agent_runs</code> ; the IT dashboard shows a verification status.
11	A	Compliance (#13) + Multi-System (#15)	Re-runs after every human action above. Checks NDA, asset return and access revocation — satisfied either by a matching completed task or an OCR-validated document. Multi-system consolidates the three stand-in sources.	<code>compliance_checks</code> rows; a compliance task left pending <i>with the exact missing items named</i> if anything is short.
12	H	Finance — GATE 2	Anfia sees the queue of cases where hr/manager/it are done but dues are not settled, and clicks "Mark dues settled". This writes through the <code>finance_mark_dues_settled</code> RPC, not a table update — see §8 for why.	<code>exit_cases.finance_cleared = true + a dues note</code> .
13	A	Finance Agent (#4)	<code>/finance-settle-check</code> re-evaluates: all three prior stages done <i>and</i> <code>finance_cleared</code> . On the pending→done transition it fires the completion notice.	Finance task → done; completion email to the employee and HR.
14	A	Risk (#12) + Rehire (#21)	Scores the case on the four weighted factors and derives rehire eligibility with a rationale. Written to HR-only columns.	<code>exit_cases.risk_score / risk_level / rehire_eligible</code> .
15	H	HR — GATE 3	Clicks "Issue relieving letter". The UI offers it only when the case is clear — and the database re-checks independently : the RLS policy's <code>WITH CHECK</code> calls <code>exit_case_cleared_for_relieving(id)</code> , which requires <code>finance_cleared</code> , zero non-done tasks across hr/manager/it/finance, and all four stages actually present. A tampered request is rejected by Postgres.	<code>relieving_letter_issued = true</code> , <code>issued_at</code> , <code>issued_by</code> , <code>status</code> → <code>completed</code> .
16	A	Email (#6)	<code>/issue-relieving-letter</code> sends the closing notice.	Relieving-letter email; the employee's dashboard switches to the exit-complete view.

Where an exit can stop, and what happens

Exception	Handling
Manager rejects the KT plan	Graph branches to escalate: an <code>exit_tasks</code> row titled "Escalated: manager rejected KT plan — HR review needed" with <code>escalation_state='open'</code> , then END. It does not auto-retry. HR sees it on the Escalations page and can re-route or resolve, each transition audited.
Compliance short an item	The compliance task stays pending with the specific reason ("NDA: pending; ..."). Finance cannot complete, so the relieving gate stays shut.
A task goes ≥ 5 days overdue	#9 composes an escalation naming the blocker, days overdue and impact, and emails the blocker. Only when someone runs it.
Agent service is not running	Frontend calls are best-effort: the RLS-scoped write still lands, the follow-on pipeline call fails silently rather than blocking the click. The exception is the manager's Sign clearance, where a failure or <code>advanced:false</code> is surfaced in the UI on purpose.
A capstone run that cannot complete	#24 returns a clear <code>{"status": "blocked", "reason": ...}</code> — a valid terminal state, not a crash and not a false success.

7. File / code map

Enough to answer "what is this file?" for anything on screen.

7.1 Agent service — /agents (Python)

Modules are grouped into packages: `hub/` (the supervisor), `spokes/` (the per-domain agents), `core/` (config, LLM, trace, notifications, calendar, prompt loading) and `analytics/` (the aggregate/reporting agents). `run_case.py`, `service.py`, `oauth_setup.py` and `e2e_test.py` stay at the `/agents` root. Paths below are relative to `/agents`.

File	What it is
<code>hub/supervisor.py</code>	The hub. The LangGraph <code>StateGraph</code> with all stage nodes and the conditional manager-gate edge. Start here when explaining the architecture.
<code>run_case.py</code>	The terminal entry point. <code>python -m agents.run_case <case_id></code> . Thin wrapper over the supervisor that streams the live trace; supplies default KT and interview text so a bare run still exercises those spokes.
<code>service.py</code>	The browser bridge. stdlib HTTP server on localhost:8787 with nine POST routes: <code>/activate-exit</code> , <code>/submit-exit-interview</code> , <code>/issue-relieving-letter</code> , <code>/validate-document</code> , <code>/execute-deprovisioning</code> , <code>/finance-settle-check</code> , <code>/manager-approve</code> , <code>/reject-manager-task</code> , <code>/escalation-audit</code> . Most end by re-running compliance for that one case.
<code>core/trace.py</code>	The <code>@traced_node</code> decorator plus <code>log_llm</code> / <code>log_db</code> . Indentation, colours, truncation, redaction, <code>TRACE_LEVEL</code> .
<code>core/config.py</code>	Loads <code>.env</code> (with <code>override=True</code>) and creates the one Supabase client, using the service key — correct here because this is server-side and never shipped to a browser.
<code>core/llm.py</code>	The only place an LLM is called. Plain HTTP <code>POST {ANTHROPIC_BASE_URL}/v1/messages</code> through Portkey; handles thinking blocks and empty-content reasoning responses; <code>ask_claude</code> / <code>ask_claude_json</code> .
<code>spokes/hr_agent.py</code>	#2 / #5 / #10 — checklist and KT review subgraphs, <code>IT_OWNED_TITLE_RE</code> , due-date derivation.
<code>spokes/it_agent.py</code>	#3 — deprovisioning plan generation, prompt tuned to the IT dashboard's keyword buckets.
<code>spokes/it_deprovisioning_agent.py</code>	#18 — generate wrapper + execute / verify / audit + <code>MockITAdapter</code> and two fault-injection adapters.
<code>spokes/finance_agent.py</code>	#4 — deterministic clearance gate, no LLM.
<code>spokes/compliance_agent.py</code>	#13 — NDA / asset / access verification, item-level tracing, <code>compliance_checks</code> upsert.
<code>spokes/risk_agent.py</code>	#12 — weighted scoring, department criticality map, thresholds.
<code>spokes/rehire_agent.py</code>	#21 — rehire eligibility with confidence and evidence.
<code>spokes/exit_intel_agent.py</code>	#8 (per-case) and #19 (longitudinal) — two subgraphs in one module.
<code>analytics/analytics_agent.py</code>	#14 — aggregate (Python) → narrate (LLM) → persist.
<code>analytics/workflow_optimizer.py</code>	#17 — bottleneck stats reusing #9's breach finder.
<code>analytics/attrition_agent.py</code>	#23 — gather → identify → narrate & persist.
<code>analytics/policy_auditor.py</code>	#22 — the three policy checks and the formatted report.
<code>analytics/sla_escalation.py</code>	#9 — <code>find_breaches</code> (a pure function reused by #17 and #22) plus compose & send.
<code>spokes/smart_routing.py</code>	#11 — approver selection by department and out-of-office.

spokes/multi_system_clearance.py	#15 — <code>SystemAdapter</code> contract and the three stand-in adapters.
spokes/doc_collection.py	#16 — required-doc list, classification, real OCR, validation rules, test-image generator.
core/notifications.py	The seven email templates and the SMTP send, with the dev-log gate.
spokes/email_drafting_agent.py	#6 — the routing/tracing/recording layer over the above.
core/calendar_booking.py	Google Calendar event creation; writes <code>exit_tasks.kt_event_id</code> .
oauth_setup.py	One-time admin script to obtain the Google refresh token. Never writes <code>.env</code> itself.
hub/e2e_automation.py	#24 — the capstone composition.
e2e_test.py	Integration test: picks a seeded employee with no case, creates one, runs the pipeline, then re-queries the exact columns each dashboard reads.
core/prompts.py	<code>load_prompt()</code> — reads each agent's system prompt from the top-level <code>/prompts</code> tree (e.g. <code>prompts/hr/checklist_system.md</code>) instead of inlining it in Python.

7.2 Edge Functions — `/supabase/functions` (Deno/TypeScript)

File	What it is
<code>ask/index.ts</code>	The RAG endpoint. <code>Embed</code> → <code>match_exit_docs</code> → <code>generate</code> → <code>{answer, sources, refused, general}</code> . Holds the system prompt, the 0.35 similarity floor, the JSON-repair parser and the refusal heuristic.
<code>submit-resignation/index.ts</code>	Creates the exit case with server-derived identity. Returns the existing case if one already exists (idempotent). Triggers no agent.
<code>forward-to-hr/index.ts</code>	Emails an unanswered question to HR on explicit click only. On SMTP failure, logs to <code>agent_runs</code> and tells the employee the truth.

7.3 Frontend — `/src`

File	What it is
<code>main.jsx / App.jsx</code>	Entry point; session handling and role resolution.
<code>routes/AppRoutes.jsx</code>	The whole route tree. Every role branch is guarded by <code>role === '...'</code> with a redirect otherwise.
<code>routes/<role>/<Role>Layout.jsx</code>	Per-role shell: sidebar, and the data fetch shared with every child page via outlet context. <code>EmployeeLayout</code> also holds the resignation gate.
<code>routes/<role>/<Role>Pages.jsx</code>	Every page for that role as a named export. HR has 11, Employee 9, IT 7, Manager 6, Finance 1.
<code>lib/supabase.js</code>	The single browser client — URL and anon key only .
<code>lib/ktScope.js</code>	<code>caseClearanceState</code> — shared by the manager dashboard card and Clearances page so both agree on whether an employee is ready to sign.
<code>lib/clearanceStatus.js, financeStatus.js, agentRunText.js, format.js</code>	Shared derivation and formatting helpers, kept out of the page files so two pages cannot disagree.
<code>components/</code>	<code>LoginPage</code> , <code>Sidebar</code> , <code>PageHead</code> , <code>EmployeeGroup</code> , <code>RoleSwitcher</code> .
<code>styles/tokens.css, dashboards.css</code>	The design tokens and the dashboard component styles. The approved design lives here.

7.4 Migrations and scripts

File	What it is
------	------------

0001_schema.sql	All base tables, indexes, the <code>vector</code> extension, and the HNSW index.
0002_rls.sql	The security model. <code>app_current_role()</code> , RLS on every table, and the first restricted views. Read this one to explain security.
0003_rag.sql	<code>match_exit_docs</code> , deliberately not <code>SECURITY DEFINER</code> .
0005, 0019, 0025	<code>it_task_view</code> and <code>manager_case_view</code> — creation and later fixes (IT's view gained a lateral join to the latest verification status).
0006, 0007, 0009, 0011	<code>analytics_insights</code> , <code>kt_reviews</code> , <code>agent_runs</code> , <code>case_documents</code> .
0013, 0014, 0027	The finance role: the two columns, <code>finance_mark_dues_settled</code> , and <code>finance_reject_dues</code> .
0017, 0018	The relieving letter: <code>exit_case_cleared_for_relieving</code> and the HR update policy that calls it.
0020_role_views_security_invoker.sql	Important. Re-asserts <code>security_invoker = false</code> on all five role views. Supabase's Advisor flags these as an error; applying its suggested fix has broken the dashboards three times. If a dashboard suddenly shows zero rows, re-run this file first.
0024	The private exit-documents Storage bucket and its policies, keyed on <code>storage.foldername(name)[1]</code> being a case the caller owns.
0028	Re-stages the three IT-owned tasks that the checklist LLM had mis-filed as manager tasks, guarded by an EXISTS so it only touches cases that already have an IT plan.
scripts/seed/seed.js, scripts/seed/seed_delegates.js	103 demo profiles, cases for Emp001–010, and one real delegate account per role. Always pass <code>DEMO_EMAIL_DOMAIN=company.com</code> .
scripts/ingest_docs.js	The RAG ingest: chunk → embed → insert. Idempotent per source.
scripts/exit_policy.md	The source corpus — the 67 chunks come from here.
scripts/verify/verify_*.cjs , e1_browser_test.cjs	Playwright verification scripts. They create disposable cases, assert against a real browser, and purge afterwards.

8. Database and security model

8.1 Tables

Table	Key columns	Purpose and access
profiles	id (FK → auth.users), role, full_name, email, employee_id, department, out_of_office	One row per user. <code>role</code> drives every policy. You may read your own row; HR may read all.
exit_cases	id, employee_id, employee_name, department, role_title, manager_id, hr_id, last_working_day, status, resignation_reason, finance_cleared, dues_note, finance_rejected, relieving_letter_issued, issued_at, issued_by · risk_level, risk_score, rehire_eligible	One row per offboarding. Exactly one SELECT policy exists, for HR. Every other role reaches it only through a restricted view.
exit_tasks	id, case_id, stage, title, status, due_date, kt_event_id, reason, escalation_state	The checklist. <code>stage</code> (hr / manager / it / finance / compliance) routes a task to a dashboard. All five roles have a scoped policy — a checklist is not sensitive.
exit_interviews	case_id, summary, sentiment, themes, rehire_eligible, rehire_reason, reason_for_leaving, feedback, would_recommend, comments	HR only. The employee's own page checks submission through a view exposing just <code>case_id</code> and <code>created_at</code> .
exit_docs	id, source, section, content, embedding vector(1536)	The RAG chunk store. RLS on, zero policies — deliberately unreadable by anyone but the service key.
case_documents	case_id, doc_type, file_path, status, validation_detail	One row per <i>submitted</i> upload. "Missing" is a set difference computed at read time, never a stored status.
compliance_checks	case_id, item, status, source, evidence, failure_reason, checked_at	Item-level compliance results (78 rows live). No migration file creates this table — see §10.
agent_runs	case_id, stage, detail, agent, status, metadata, created_at	The durable agent log (323 rows) behind HR → Agent activity, and the audit trail / idempotency guard for #18. agent, status, metadata exist live but not in migration 0009.
kt_reviews	case_id, summary, gaps, complete	Evaluative KT findings. HR and the owning manager only — deliberately not in <code>exit_tasks.title</code> , which the employee reads verbatim.
analytics_insights	narrative, stats, agent_type	Narrative outputs from #14 / #17 / #22 / #23, separated by <code>agent_type</code> . HR only.
trend_alerts	theme, department, severity, detail	Rising-theme alerts from #19. HR only.

8.2 The security model, and why it is built this way

The frontend holds the **anon key**, which any user can read out of the JavaScript bundle. So the frontend cannot be where permissions are decided. Postgres is.

Layer 1 — default deny

RLS is enabled on every table. A table with RLS on and no matching policy returns **zero rows** and rejects writes. Nothing is visible unless a policy says so.

Layer 2 — one role helper

`app_current_role()` returns the caller's role. It is `SECURITY DEFINER` so it can read `profiles` while `profiles` is itself under RLS, and it sets `search_path = ''` with fully-qualified names — so an attacker cannot shadow `profiles` with an object in a schema they control and have the definer function resolve to it. Execute is granted to authenticated only.

Layer 3 — the HR-only columns

`exit_cases` has **one** SELECT policy, for HR. There is no employee policy and no manager policy — deliberately. Adding one would hand that role the whole row, including `risk_score`, `risk_level` and `rehire_eligible`.

Layer 4 — restricted views as the only other way in

Five views run with `security_invoker = false`, i.e. as the view *owner*, so they can see past that HR-only policy — and their fixed SELECT list decides what the caller actually gets. Each view self-filters on `auth.uid()`.

View	For	Exposes
<code>employee_exit_view</code>	Employee	<code>id</code> , <code>employee_name</code> , <code>department</code> , <code>role_title</code> , <code>last_working_day</code> (+ relieving fields)
<code>manager_case_view</code>	Manager	Same safe columns, filtered to <code>manager_id = auth.uid()</code>
<code>it_task_view</code>	IT	<code>stage='it'</code> tasks + employee name/department + the latest verification status via a lateral join
<code>finance_case_view</code>	Finance	Safe columns + <code>finance_cleared</code> + <code>dues_note</code>
<code>employee_interview_status_view</code>	Employee	<code>case_id</code> + <code>created_at</code> only — enough for "have I submitted?", nothing more

None of these views may ever gain a risk or rehire column. That is stated in a comment on each view in the migration.

The Supabase Advisor warning you will be asked about. Supabase's Advisor reports "Security Definer View" as an *ERROR* on all five views and suggests setting `security_invoker = on`. **That suggestion is wrong for this schema and has broken the dashboards three times.** With invoker semantics the view runs as the caller, who has no policy on `exit_cases` — so it returns zero rows and the dashboard goes blank. The views are safe precisely because their column list is fixed and they self-filter on `auth.uid()`. The fix for a zero-row dashboard is to re-run `0020_role_views_security_invoker.sql`.

Layer 5 — write paths are narrower than read paths

- **Finance writes through an RPC, not an UPDATE policy.** `exit_cases` has no SELECT policy for finance by design. Postgres cannot apply an UPDATE's `WHERE` clause to a row the caller cannot see — so a plain RLS UPDATE policy silently matches zero rows. Hence `finance_mark_dues_settled: SECURITY DEFINER`, checks the caller's role itself, and forward-only (always sets `finance_cleared = true`). This is a genuinely non-obvious interaction and a good thing to be able to explain.
- **Column-level grants.** `GRANT UPDATE (finance_cleared, dues_note)` — an authenticated caller can write those two columns and nothing else.
- **Forward-only approvals.** The finance UPDATE policy's `WITH CHECK` pins `finance_cleared = true`, so the surface cannot be used to *un*-clear a case.
- **The relieving gate is re-checked in the database.** The HR update policy's `WITH CHECK` calls `exit_case_cleared_for_relieving(id)`, which requires `finance_cleared`, zero non-done tasks across all four stages, and all four stages present. Bypassing the UI does not bypass this.
- **Storage is scoped by path.** The bucket policy checks `storage.foldername(name)[1]` is a case the caller owns, so an employee cannot upload into, or read, another person's folder.
- **Identity is never taken from the request body.** Both writing Edge Functions derive the employee from the caller's own JWT.

Layer 6 — key discipline

Key	Where it lives	Rule
Anon key	Browser bundle (VITE_ prefixed)	The only key the frontend ever holds. RLS constrains everything it can do.
Service key	Python agent service; Edge Functions	Bypasses RLS. Server-side only, never in the bundle, never committed.

Portkey, Gmail app password, Google refresh token	.env (gitignored) and Supabase Edge Function secrets	Deployed functions do not read the local .env — the same secret often has to be set in two places.
---	--	---

9. The five roles

Role	Demo account	Pages	Can do	Cannot see
Employee	Emp001– Emp100 <EmpId>@	Dashboard, My exit, Tasks, Documents, Knowledge transfer, Exit interview, Timeline, Help	Submit resignation (forced first, via the gate); complete their own tasks; upload documents; submit the exit interview; ask the RAG assistant and forward an unanswered question to HR.	Their own risk score, rehire eligibility, interview sentiment or summary; the evaluative KT review; anyone else's case.
HR	Siva siva@company.com	Dashboard, All exits, Escalations, Risk & compliance, Exit interviews, Trends, Clearances, Policy audit, Reports, Agent activity, Settings	See every case and every assessment field; work escalations (re-route / resolve, each transition audited); issue the relieving letter once the DB agrees the case is clear; read agent activity and the analytics narratives.	Nothing — HR is the only role with a SELECT policy on <code>exit_cases</code> .
Manager	Aravidhan aravidhan@company.com	Dashboard, My team, Exiting reports, KT approvals, Clearances, Timeline, Help	Gate 1. Review or reject each KT task; sign one clearance per employee, which opens the IT stage. Rejection creates an escalation for HR.	<code>risk_level</code> / <code>risk_score</code> / <code>rehire_eligible</code> ; interview summary and sentiment; any case where they are not <code>manager_id</code> .
IT	Aswin	Dashboard, Deprovisioning, Asset recovery, Access reviews, Approvals, Audit log, Help	Review and approve the generated deprovisioning plan; approval triggers the execute/verify/audit pass and shows a verification status per task.	Everything except <code>stage='it'</code> tasks — the IT policy is literally <code>role = 'it' AND stage = 'it'</code> .
Finance	Anfia anfiacj@gmail.com	Dashboard (a work queue), Help	Gate 2. See cases where hr/manager/it are done but dues are unsettled; "Mark dues settled" (via RPC) or reject dues. Clearing fires the completion email.	Assessment columns — verified in a real browser: a direct read of <code>exit_cases</code> as Anfia returns zero rows.

Demo passwords are deterministic and apply to fake seed data only — never production. Note for the demo: a duplicate `@company.com` (misspelled) account pair was created by an accidental seed run and deleted on 2026-09-14; `@company.com` is the real data and owns all seeded cases.

10. Honest status — what is real, manual, or mocked

10.1 Verified working end to end

- Schema, seed data (107 profiles), RLS and all five restricted views — including a browser-verified denial test for the finance role.
- RAG assistant: deployed Edge Function, 67 real chunks, citations, scope classification, forward-to-HR.
- The supervisor hub and all clearance spokes, with a live terminal trace showing LLM model, latency and token counts per call.
- Real Google Calendar events with the event id written back to the task row.
- Real Gmail SMTP delivery (currently enabled).
- Real Tesseract OCR against an uploaded document, flipping `case_documents` to `validated`.
- The full browser path for a case: resignation → HR checklist → manager KT approval → IT plan appears → IT approves → finance settles → compliance clears → HR's "Issue relieving letter" becomes available. Verified by `scripts/verify/verify_manager_gate.cjs` with disposable cases, then purged.
- Capstone #24, both branches: a completed run and a `--reject` run that exercised escalation and ended in a clean blocked state rather than a crash.

10.2 Works, but only when a human runs it — no scheduler exists

I searched the entire repository: there is no cron, no `pg_cron`, no GitHub Actions workflow (`.github/` does not exist), and no timer in any Python module. Six agents whose briefs say "periodically", "weekly", "monitors", or "on scheduled triggers" are run by hand: **#9 SLA escalation**, **#14 dashboard insights**, **#17 workflow optimizer**, **#19 trend analyst**, **#22 policy auditor**, **#23 predictive attrition**. Each has a working `python -m agents.<name>` entry point and has produced real persisted output — the arithmetic and the writes are genuine, the *automatic recurrence* is not built. Adding it is a small job (a `pg_cron` job or a Windows scheduled task calling the existing entry points), just not done.

10.3 Mock or demo-scale

Item	What is real	What is a stand-in
#15 Multi-System Clearance	The adapter contract (enforced timeouts, retries, status codes, error capture), and three live queries returning real consolidated status.	The "three external systems" are our own Supabase tables. No ITAM / HRMS / finance platform exists to integrate with.
#18 execute step	Plan generation, human approval, independent verification, per-task audit row, idempotency guard, fault-injection tests.	<code>MockITAdapter</code> — no Okta / Jamf / Workspace call. Nothing is actually disabled.
Google Calendar	Real API, real events, real event ids persisted.	One shared demo account holds every booking; no per-user calendars. Production would use a service account.
#10 KT Document Reviewer	The LLM review, the gap tasks, the <code>kt_reviews</code> rows.	No KT document upload exists in the UI. The employee's Knowledge-transfer page is a read-only task list; document text is supplied from a file path on the CLI.
#12 tenure input	The weighting, thresholds and the other three inputs.	No <code>hire_date</code> column exists; tenure is proxied from <code>profiles.created_at</code> , so every seeded case reads as short tenure.
#11 department matching	Out-of-office routing to a real delegate, verified both directions.	No HR/manager/IT profile has a department set, so that branch never fires today.
#23 attrition granularity	Real signals, real persisted narrative.	Risk is identified at department level, not per named individual.

10.4 Schema drift — the live database is ahead of the migration files

These objects exist and are in daily use in the live project, but no file in supabase/migrations/ creates them. A database rebuilt purely from migrations would therefore be incomplete. Worth owning before someone else spots it:

- **compliance_checks** — the whole table (78 rows), written by #13.
- **agent_runs.agent, .status, .metadata** — migration 0009 defines only case_id, stage, detail, created_at, but it_task_view joins on metadata->>'task_id', so the live schema is required for IT's dashboard to work.
- **exit_tasks.reason and .escalation_state** — used by the escalation flow; 0026 backfills escalation_state but nothing creates it.
- **Migration 0023 is absent** from the sequence (0022 → 0024).
- **agents/requirements.txt** omits pytesseract and Pillow, which the OCR agent imports.

None of this breaks the running system — it is a reproducibility gap, not a runtime bug.

10.5 Open items

Item	Status
E1	The single complete real-browser end-to-end review across all five roles is not signed off . Individual flows are Playwright-verified; Phase 9 remains unticked in blueprint1.md on purpose, because this project's rule is that a box is ticked only after a human confirms in a real browser.
Scheduler	Named follow-up for the six manual agents (§10.2).
KT upload	The UI affordance for #10 does not exist yet.

If you are asked one hard question, it will probably be one of these.

- "Is it really multi-agent, or one script?"** — It is a LangGraph StateGraph with seven nodes and a real conditional edge; each spoke is its own compiled subgraph in its own module, individually runnable from the CLI, and agent_runs holds 323 real stage transitions from past runs.
- "Which parts are actually automated?"** — Everything between the human gates. The three gates (manager sign-off, finance dues, HR relieving letter) are deliberately human, and the IT agent never marks its own work done. That is a design posture, not a missing feature: *an agent proposes, a human disposes*.
- "Why doesn't the LLM compute the analytics numbers?"** — Because a model doing arithmetic will eventually produce a number that does not match the database, in a report HR acts on. Every count is Python or SQL; the model only writes the sentence around it.
- "Can a manager see the risk score?"** — No, and not because the UI hides it. There is no SELECT policy for managers on exit_cases at all; their view exposes five safe columns. The denial was tested directly against the database for the finance role and returns zero rows.

11. LLM vs. deterministic agents — who calls the AI

Nine of the 24 numbered roles call the language model. The other fifteen (including one human role) are deterministic Python — arithmetic, regex, or database queries. The split is deliberate: wherever the answer is a fixed rule or a number, a model is not used, because a model doing exact checks will eventually produce the wrong answer in a way that is hard to detect.

11.1 Agents that call the LLM

#	Agent	Prompt file	What it asks the AI to do
2 / 5	HR checklist generator	prompts/hr/checklist_system.md	Given the employee's role and department, generate a JSON object with <code>hr_tasks</code> and <code>manager_tasks</code> — 2–4 short imperative task titles each, tailored to the role. Manager tasks must be knowledge-transfer items only; IT and finance work is explicitly out of scope.
10	KT Document Reviewer	prompts/hr/kt_review_system.md	Read a handover document and return JSON: <code>complete</code> (boolean), <code>gaps</code> (a list of {topic, detail} objects), <code>summary</code> (2–3 evaluative sentences for the manager).
3	IT deprovisioning planner	prompts/it/deprovisioning_system.md	Generate JSON with <code>tasks</code> : 3–6 imperative deprovisioning task titles covering account revocation and asset collection, tailored to the employee's role. Titles must use concrete words (laptop, SSO, repo) so the IT dashboard's keyword-based sorting works correctly.
8	Exit interview summarizer	prompts/interview/summary_system.md	Analyse a free-text exit interview transcript and return JSON: <code>summary</code> (2–3 sentences), <code>sentiment</code> (positive/neutral/negative), <code>themes</code> (1–5 short theme strings), <code>rehire_eligible</code> (boolean), <code>rehire_reason</code> (one sentence).
7	RAG /ask assistant	Inline in supabase/functions/ask/index.ts	Given retrieved policy context and the employee's question, return JSON: <code>scope</code> (policy / general / refused), <code>answer</code> (2–3 sentences strictly from context for policy, general guidance for HR-adjacent questions, a refusal for off-topic ones), <code>sections</code> (exact section titles used). Must not invent company-specific facts (notice periods, amounts) when scope is "general".
14	Analytics / dashboard insights	prompts/analytics/dashboard_insights_system.md	Given a precomputed stats dictionary, write a 3–4 sentence narrative for the HR insights panel calling out bottlenecks and attrition signals. Plain prose only — the prompt explicitly says "do not do any arithmetic yourself."
17	Workflow optimizer	prompts/analytics/workflow_optimizer_system.md	Given precomputed bottleneck stats, identify the biggest pipeline delays and propose 2–3 concrete workflow reconfigurations to reduce them. Plain prose, no arithmetic.
22	Policy compliance auditor	prompts/analytics/policy_audit_system.md	Given a precomputed list of breaches (SLA overruns, missing approvals, skipped steps), write 3–5 sentences: overall policy health, most common breach type, one concrete process fix. Must not invent breaches.
23	Predictive attrition analyst	prompts/analytics/attrition_system.md	Given a list of at-risk departments with their signals (already identified by Python), write 3–5 sentences of concrete retention interventions per department. Must not add departments or do arithmetic itself.

11.2 Agents that use no LLM — and why

#	Agent	What it does instead, and why no LLM
1	Manager	A human dashboard role, not an agent. No automation here by design — the manager approval is a deliberate human gate.
4	Finance clearance	Checks two boolean conditions: all prior stages done AND <code>exit_cases.finance_cleared = true</code> . The answer is exact and binary; a model adds only cost and potential variance.
6	Email drafting (notifications)	Composes fixed transactional emails from structured templates in <code>core/notifications.py</code> . These are notices, not correspondence — the content is the same every time. A model would add latency and unpredictability for zero value.
9	SLA escalation	Date arithmetic: <code>(today - due_date).days ≥ 5</code> . Pure counting and comparison. The email itself is also templated.
11	Smart routing	Selects the right approver by checking <code>profiles.out_of_office</code> and falling back to a delegate. Deterministic lookup over known fields — no judgment required.
12	Risk scoring	Weighted formula: <code>score = tenure×0.2 + role_criticality×0.3 + sentiment_risk×0.3 + tasks_incomplete_fraction×0.2</code> . This is intentionally arithmetic — an LLM producing a risk score that does not match the formula would undermine trust in the number HR acts on.
13	Compliance verification	Regex keyword match over exit task titles: asset return (<code>laptop device asset ...</code>), NDA (<code>\bnda\b non-disclosure ...</code>), access revoked (<code>sso account revoke ...</code>). Also accepts an OCR-validated document row as evidence. Exact-match rules; a model would add hallucination risk for checks that must be provably correct.
15	Multi-system clearance	Queries three database adapter objects that return real status codes and evidence strings. Aggregation is pure Python. No language generation involved.
16	Document collection	Downloads the uploaded file, runs real Tesseract OCR to extract text, then checks for required keyword groups plus the employee's own name and a date. An LLM is not used because the validation rules are exact substring checks — a model reading an image might hallucinate pass or fail.
18	IT execute/verify/audit	Calls the adapter, re-derives the expected action independently, compares — three steps, all deterministic. The <code>MockITAdapter</code> currently backs it; production would be Okta/Jamf API calls, not LLM calls.
19	Longitudinal trend	Python <code>Counter</code> over all interview themes. Counts themes, identifies those appearing ≥ 2 times, writes alert rows. The counting is exact; if the result needed a narrative it would call the LLM, but trend detection itself is arithmetic.
20	Supervisor / orchestrator	Walks the case through the LangGraph graph: routes to each spoke in order, evaluates the manager-gate condition (<code>approved → IT</code> , <code>rejected → escalate</code>), and records transitions. Graph routing requires no language generation.
21	Rehire assessment	Reads <code>exit_interviews.rehire_eligible</code> if the interview exists, otherwise derives it from <code>risk_level ≠ "high"</code> . One boolean from existing data — wraps <code>risk_agent.score()</code> with no new reasoning.
24	E2E automation	Composes the existing pipeline agents in one script. Makes no LLM calls of its own; it just orchestrates what the individual spokes already do.

12. The actual system prompts

Each prompt file is the complete instruction given to the language model for that agent — nothing is added to it at runtime except the user message (the data). The files live in `/prompts/` and are loaded by `agents/core/prompts.py`'s `load_prompt()`. The RAG assistant's prompt is inlined directly in the Edge Function.

#2 / #5 — HR checklist generator

File: `prompts/hr/checkList_system.md`

You generate an offboarding checklist for an exiting employee. Return ONLY a JSON object with keys 'hr_tasks' and 'manager_tasks', each a list of 2-4 short imperative task titles tailored to the employee's role and department. Titles must be neutral and actionable (things to do), never evaluative -- they are shown to the employee as their own to-do list. 'manager_tasks' are KNOWLEDGE-TRANSFER and handover items only: documenting, transferring ownership, briefing the team, handing off contacts, scheduling a handover session. Never put IT deprovisioning (revoking or disabling access, accounts, SSO, credentials, repository permissions; collecting laptops, devices or access cards) or finance settlement work in 'manager_tasks' -- separate IT and Finance agents own those stages and generate their own plans.

Note: the Python agent drops any title the model misfiles into `manager_tasks` that matches `IT_OWNED_TITLE_RE` (a regex covering revoke, disable, SSO, repo, laptop, etc.). A real bug — the early model putting "Revoke GitHub access" in the manager list — motivated both the guard in the prompt and the post-processing filter.

#10 — KT Document Reviewer

File: `prompts/hr/kt_review_system.md`

You review a knowledge-transfer handover document for completeness ahead of an employee's exit. Return ONLY a JSON object with keys: complete (boolean), gaps (a list of {topic, detail} objects, empty if none), summary (2-3 sentences for the employee's manager, evaluative framing is fine here).

#3 — IT deprovisioning planner

File: `prompts/it/deprovisioning_system.md`

You generate an IT deprovisioning plan for an exiting employee. Return ONLY a JSON object with key 'tasks': a list of 3-6 short imperative task titles covering account/access revocation and asset collection, tailored to the employee's role and department. Use concrete words so each task sorts correctly in a checklist: for asset items mention 'laptop', 'access card' or 'device'; for identity/account access say 'SSO' or 'account'; for developer/source access say 'repo' or 'git'; for any other application access, name the application.

The concrete-word instruction exists because `it_task_view` and the IT dashboard bucket tasks by keyword — a vague title like "Remove access" would not sort into the correct category.

#8 — Exit interview summarizer

File: `prompts/interview/summary_system.md`

You analyze an employee exit interview transcript. Return ONLY a JSON object with keys: summary (2-3 sentences), sentiment (one of 'positive', 'neutral', 'negative'), themes (a list of 1-5 short theme strings, e.g. 'compensation', 'management', 'career growth'), `rehire_eligible` (boolean), `rehire_reason` (one sentence).

The output is HR-only. The employee sees only "submitted" (timestamp only, via a restricted view). Risk scoring agent (#12) consumes `sentiment`; rehire agent (#21) consumes `rehire_eligible`.

#7 — RAG /ask assistant

Inline in `supabase/functions/ask/index.ts` (the Edge Function)

```
You are the ExitAI assistant, helping an employee through the company exit process.

Reply with ONLY a JSON object, no markdown fences, with exactly these three keys:

"scope": one of "policy", "general", "refused".
  "policy" - the provided context answers the question.
  "general" - the context does NOT answer it, but the question is about work, employment,
              HR, or leaving a job, so general guidance is useful.
  "refused" - the question has nothing to do with work, employment, HR, or exiting a job.

"answer": 2-3 plain sentences addressed to the employee. Never put citations, section titles,
          section numbers, or a "Sections used" line in this field.
          If scope is "policy": answer strictly from the context.
          If scope is "general": give genuinely useful general guidance, and NEVER state a
              company-specific fact as if it were this company's rule -- no notice periods, settlement
              timelines, deadlines, amounts, or entitlements. Say those depend on their contract and HR.
          If scope is "refused": one short sentence saying this isn't something you can help with,
              and to contact HR.

"sections": an array of the section titles you used, copied exactly as they appear in
            square brackets in the context. Must be [] unless scope is "policy".
```

The user message sent to the model is: *Context: [4 retrieved policy chunks] \n\nQuestion: [employee's question]*. The similarity floor (0.35 cosine) means that if the top retrieved chunk is not actually relevant, the answer is labelled "general" or "refused" and the chunks are not cited. A "Forward to HR" button appears whenever `refused = true` or the answer begins "I don't have..."

#14 — Analytics / dashboard insights

File: `prompts/analytics/dashboard_insights_system.md`

```
You are an HR analytics assistant. Given aggregate offboarding stats (already computed --
do not do any arithmetic yourself), write a short (3-4 sentence) narrative for an HR insights
panel calling out bottlenecks and attrition signals. Plain prose, no markdown, no restating
the raw numbers verbatim.
```

#17 — Workflow optimizer

File: `prompts/analytics/workflow_optimizer_system.md`

```
You are an HR operations analyst. Given aggregate offboarding bottleneck stats (already
computed -- do not do any arithmetic yourself), identify which pipeline stages and departments
are the biggest bottlenecks and propose 2-3 concrete workflow reconfigurations to reduce delay.
Plain prose, no markdown, no restating the raw numbers verbatim.
```

#22 — Policy compliance auditor

File: `prompts/analytics/policy_audit_system.md`

```
You are a compliance auditor summarizing a policy audit of offboarding cases. You are given
the exact breaches already found (already computed -- do not do any arithmetic or invent
breaches yourself). Write 3-5 sentences: overall health, which breach type is most common,
and one concrete process fix. Plain prose, no markdown.
```

#23 — Predictive attrition analyst

File: `prompts/analytics/attrition_system.md`

You are an HR retention analyst. Given a list of at-risk departments with the signals that flagged them (already identified -- do not do any arithmetic or add departments yourself), write 3-5 sentences of concrete retention interventions per department. Plain prose, no markdown.

Pattern visible across every analytics prompt. Every analytics system prompt contains an explicit "already computed — do not do any arithmetic yourself" instruction. This is a deliberate guard: a model that recomputes numbers from prose will occasionally hallucinate a figure that does not match the database, and HR or Finance acts on those figures. The model's job is to write the English around numbers Python has already verified.

13. Q&A — 27 questions a manager or director might ask

Plain-language answers to the questions most likely to come up in a review or demo. Questions already answered in §10's callout are repeated here for completeness.

The AI and what it does

1. Is it really multi-agent, or just one script with a few functions?

It is a LangGraph `StateGraph` with seven stage nodes and a real conditional edge (approve → IT path, reject → escalation path). Each spoke is its own compiled subgraph in its own Python module, individually runnable from the CLI. `agent_runs` holds 323 real stage transitions from past runs, one row per agent invocation.

2. How many agents actually call the AI model?

Nine of the 24 numbered roles. The other fourteen are deterministic Python — arithmetic, regex keyword matching, date comparison, or database queries. See §11 for the full breakdown.

3. How is the LLM prompted? Does it see sensitive employee data?

For the checklist, the model receives only the employee's *role title* and *department* — not their name, resignation reason, or salary. For the interview summarizer, it receives the full free-text interview transcript. For the RAG assistant, it receives retrieved policy text plus the employee's typed question. The model never sees passwords, keys, or any data from other employees' cases.

4. What happens if the AI generates a wrong or poor checklist?

Two guards exist. First, the Python agent drops any task the model misfiles into the wrong category (e.g. "Revoke GitHub access" appearing in the manager list) using a regex filter. Second, the checklist is reviewed by the employee and HR — a generated task that does not make sense for the role will be caught before it matters. The checklist is a starting point, not an approved instruction.

5. Why do some agents not use AI at all?

When the answer is a fixed rule or a number — "are all prior stages done?", "does this file contain the word NDA?", "how many days overdue?" — a model adds latency, cost, and hallucination risk without adding value. Every place where an LLM was deliberately not used is called out in §11.2 with the specific reason.

6. What if the AI gives a hallucinated answer in the RAG chatbot?

Two containment layers. First, the prompt instructs the model to answer only from the retrieved context for company-specific facts, and to refuse to state a notice period or settlement amount it cannot find in context. Second, if the model still produces a "don't have" phrasing, the code classifies it as a refusal and shows the employee a "Forward to HR" button instead of displaying the answer as company policy. The most dangerous failure — a confidently stated wrong notice period — is the case the prompt is designed to prevent.

7. Can the AI refuse to process an exit case?

No. The only agent that calls the model for case processing is the checklist generator and the IT planner — both are triggered automatically and both produce JSON. If the model returns malformed JSON or errors, the Python code raises an exception and the case stays in its current state. The human gates (manager, finance, HR) still function regardless.

Privacy and data

8. What data does ExitAI store about an employee?

Profile (name, email, department, role), exit case (last working day, resignation reason, manager/HR assigned, clearance status), tasks (checklist items and their status), uploaded documents (stored in a private Supabase Storage bucket), and the exit interview (free-text responses). For HR only: the LLM-generated interview summary, sentiment, themes, rehire eligibility, and risk score. Nothing about current (non-exiting) employees beyond what exists in `profiles` for login purposes.

9. Can an employee see their own risk score or interview feedback?

No. `risk_score`, `risk_level`, `rehire_eligible`, and the interview `summary` / `sentiment` / `themes` are stored in columns that have exactly one SELECT policy in the database: HR only. The employee's view of their case exposes five safe columns (name, department, role, last working day, relieving letter status) and no assessment fields. This was tested directly: logging in as an employee and querying the table returns zero assessment-column data.

10. How is the employee's privacy protected between roles?

Row Level Security in Postgres enforces it, not the UI. A manager can only see cases where they are the assigned `manager_id`. IT sees only stage='it' tasks for active cases. Finance sees safe columns plus the clearance flag — it cannot read assessment fields even with a direct database query. The frontend holds only the anon key (public, RLS-constrained); no role can see another role's data because there is no policy granting it.

11. What happens to an employee's data after they are cleared?

The rows remain in the database — the case status moves to "completed", `relieving_letter_issued` becomes true, and the rows are auditable. There is no automated deletion or anonymisation built yet. In a production deployment you would add a data-retention policy (e.g. archive after N years), but that is out of scope for this build.

The mock and manual parts

12. Which parts are actually automated end to end?

Everything between the human gates fires without anyone clicking anything: checklist generation, KT calendar booking, OCR document validation, interview summarisation, IT plan generation, compliance re-checking after every action, finance clearance evaluation, risk scoring, and the relieving-letter email. The three deliberate human gates are the manager's KT sign-off, the finance dues settlement, and HR's issue-relieving-letter click. The IT agent also never marks its own tasks done — IT staff approve each item individually.

13. What would it take to make the mock IT execution part real?

In `agents/spokes/it_deprovisioning_agent.py`, replace `MockITAdapter` with an adapter that calls the real provider (Okta's `/api/v1/users/{id}/lifecycle/deactivate`, Jamf's device management API, or Google Workspace's Admin SDK). The verification logic, the audit row, and the error handling already exist and work — only the one `_execute()` method changes. Estimated: a day per provider, plus integration testing.

14. What would it take to make the multi-system clearance real?

Override `_fetch()` on each of the three adapter classes (`HRMSAdapter`, `ITAMAdapter`, `FinanceAdapter`) to call the real external API instead of querying the local Supabase table. The contract — timeouts, retries, status codes, error capture — is already built. You would need API credentials for each system and a mapping from Supabase employee IDs to the external system's user IDs.

15. The six "periodic" agents — how do they actually run?

By hand, from the command line, whenever someone remembers to run them. There is no scheduler, cron job, or GitHub Action anywhere in the repo. Adding recurrence is a small job — a `pg_cron` entry or a system-level scheduled task calling the existing `python -m agents.<name>` entry points — but it is not built. This is stated plainly in §10.2.

Building and architecture

16. What does "hub and spoke" mean in practice?

The Supervisor (#20) is the hub — one LangGraph graph that owns the case state and calls each domain agent in sequence. Each domain agent (HR, IT, Finance, Compliance, Risk) is a spoke — its own compiled subgraph that can run standalone from the CLI and knows nothing about the other spokes. Spokes never call each other; everything goes through the hub. This means adding a new agent is adding a new spoke and wiring it into one place in `supervisor.py`.

17. How would you add a new agent to this system?

Four steps: (1) Write a new subgraph in `agents/spokes/` or `agents/analytics/` using the same `StateGraph` pattern. (2) If it uses the LLM, add a prompt file under `prompts/` and load it with `load_prompt()`. (3) Wire it into `supervisor.py` by importing the subgraph and adding one node and one edge. (4) Add an entry point to `service.py` if the frontend needs to trigger it. The tracing, DB logging, and email routing are all in shared core modules — the new agent just calls them.

18. How is this different from just a spreadsheet or a shared task list?

Three things a spreadsheet cannot do: (1) generate a role-specific checklist automatically when an employee resigns, (2) enforce the clearance sequence in the database so a relieving letter cannot be issued even if someone tries to skip steps, and (3) surface escalations automatically when tasks go overdue — naming the specific blocker and downstream impact. A spreadsheet also cannot gate the IT deprovisioning plan on manager approval, or verify that an uploaded document actually contains an NDA signature.

19. What is the cost per exit, roughly?

The LLM is called nine times per case during a normal end-to-end run: checklist (1), IT plan (1), interview summary (1), and analytics/audit narratives (up to 4, run manually). At current Azure OpenAI pricing through Portkey, each call processes a few hundred tokens in and a few hundred out. The total per-case LLM cost is well under \$0.10 at demo scale. The dominant cost in production would be Supabase's compute and storage tier, not the model calls.

20. How would this scale to 1,000 exits per month?

The bottleneck would be the Python agent service — currently a single-threaded `stdlib` HTTP server on one developer's laptop. For production you would containerise it (it has no shared state between requests) and put it behind a load balancer, or replace it with Supabase Edge Functions. The database (Supabase Postgres) and the LLM calls (through Portkey) are already horizontally scalable. The six manual analytics agents would need the scheduler that is currently missing.

The build itself

21. What did you build vs. what did the AI tool do?

I designed the architecture, wrote every agent, wrote every migration, built all five dashboards, and debugged everything. The AI tool (Claude) was used as a coding assistant — it suggested code given my instructions, which I reviewed, modified, and tested. Every design decision — which agents use the LLM, how the security model works, which parts are deterministic, the hub-and-spoke topology — was made by me. The AI did not design the system; I used it the way you would use a fast autocomplete that can write a LangGraph node when you describe what it should do.

22. How long did this take to build?

Roughly [the builder should fill in their own timeline here — this field intentionally left blank to avoid a number that becomes inaccurate over time]. The project spans 24 agents, 5 role dashboards, 28 database migrations, 3 Edge Functions, and end-to-end Playwright verification scripts. The hardest part was not writing the code — it was understanding the cascade effects of Supabase's `security_invoker` setting and getting the RLS model right so each role can only see exactly what they should.

23. What was the hardest bug to fix?

The security-invoker view drift. Five Postgres views must run as their *owner* (not the caller) so they can see past HR-only RLS and expose a safe column subset. Supabase's own Advisor repeatedly recommended flipping them to `security_invoker = on` — the correct setting for most views, but wrong here — and applying that recommendation wiped every non-HR dashboard. The fix is migration `0020`, which re-asserts `security_invoker = false`; the CLAUDE.md non-negotiables section explicitly says never apply the Advisor's suggestion for these five views, because this bug has occurred three times.

24. What happens if the model provider (Azure OpenAI via Portkey) is unavailable?

The checklist and IT plan generation fail with an HTTP error. The Python service returns a 500 to the frontend. The case row and any already-written task rows remain intact — nothing is corrupted. The human gates (manager, finance, HR) still function because they write directly to Supabase through the frontend, bypassing the agent service entirely. A resume is as simple as re-triggering the failed action once the model is back.

25. Is there any risk the AI misclassifies someone's exit as "high risk" incorrectly?

Yes, and it is disclosed. The risk score is a weighted formula over four inputs — three of which are real (department criticality, interview sentiment, task completion fraction) and one of which is a known proxy (tenure is derived from `profiles.created_at` because no `hire_date` column exists). Because all seed profiles were created recently, every case currently reads as short tenure and gets the maximum tenure-risk weight. The score is visible to HR only, as one signal among several — not an automated decision. A reviewable number that is slightly wrong is better than a hidden one.

26. Can you demonstrate it live?

Yes — the system is running against a live Supabase project with 107 demo profiles and 27 exit cases. The agent service needs to be started locally (`PYTHONIOENCODING=utf-8 python -m agents.service`) alongside the Vite dev server (`npm run dev`). Five demo accounts exist (see §9), each with a different role. A resignation-to-relieving-letter run takes about 10 minutes of clicking through the five dashboards with the terminal trace visible in a side pane.

27. What would a production version need that this demo does not have?

A scheduler for the six manual analytics agents. Real IT execution adapters (Okta/Jamf/Google Workspace) replacing the mock. Real external system adapters for the multi-system clearance. A `hire_date` column for accurate tenure scoring. A data-retention/deletion policy. The KT document upload UI (currently CLI-only). And the single complete five-role browser review (Phase 9, still open). Everything else — the agents, the security model, the database, the email/calendar integrations, the RAG assistant — is built and runs against real data.